we’re definitely not choosing the best possible actions. Instead, we want to use a policy that is not a uniform distribution over the actions, but is the probability distribution that would produce the maximum rewards when sampled. That is, the policy is what determines observed rewards, and the value function is a reflection of observed rewards.

In our first *n*-armed bandit problem, you were introduced to state-action-value functions. These functions often go by the name *Q function* or *Q value*, which is where deep Q-learning comes from, since, as you’ll see in the next chapter, deep learning algorithms can be used as Q functions.

Table 2.8. The action-value (Q) function

Math	English
Q_π: $(s\|a) \rightarrow E(R\|a,s,\pi)$,	Q_π is a function that maps a pair, (s, a), of a state, s, and an action, a, to the expected reward of taking action a in state s, given that we’re using the policy (or “strategy”) π.

In fact, we sort of implemented a deep Q-network to solve our contextual bandit problem (although it was a pretty shallow neural network), since it was essentially acting as a Q function. We trained it to produce accurate estimates of the expected reward for taking an action given a state. Our policy function was the softmax function over the output of the neural network.

We’ve covered many of the foundational concepts in reinforcement learning just by using *n*-armed and contextual bandits as examples. We also got our feet wet with deep reinforcement learning in this chapter. In the next chapter we’ll implement a full-blown deep Q-network similar to the algorithm that DeepMind used to play Atari games at superhuman levels. It will be a natural extension of what we’ve covered here.

Summary

- State spaces are the set of all possible states a system can be in. In Chess, this would be the set of all valid board configurations. An